# Synchronous FIFO Verification Plan

## 1. Feature Extraction & Stimulus Plan

- **Sanity/Smoke Test:** Write a few transactions without reading, then read them back until empty. Verifies basic pointers  and data integrity.

- **Fill and Drain (Full/Empty Boundaries):** Continuously write until the full flag is asserted (count == DEPTH) , then continuously read until the empty flag is asserted (count == 0).

- **Concurrent Read/Write Stress:** Assert wr_en and rd_en simultaneously across different fill levels (empty, mid-range, full) to exercise the unified count logic (2'b11).

- **Overflow Attempt:** Attempt to write data when the full flag is already high and rd_en is low. The design should ignore the write and pointers should remain stable.

- **Underflow Attempt:** Attempt to read data when the empty flag is already high and wr_en is low. The design should ignore the read and pointers should remain stable.

- **Randomized Back-to-Back:** Fully randomized delays between writes and reads to stress the pointer arbitration logic over an extended period.

- **Asynchronous Reset:** Assert rstn in the middle of active reads/writes, at full capacity, and at empty capacity to ensure pointers and count zero out immediately.

## 2. Checking Strategy (Scoreboard & Assertions)

### Assertion Checks (White-box & Protocol)

- **State Validity:** Ensure full correctly asserts when count == DEPTH , and empty asserts when count == 0.

- **Data Stability:** Verify that dout remains perfectly stable if the FIFO is not empty and no read is requested.

- **Pointer Protection:** Ensure the read/write pointers do not increment during overflow/underflow conditions.

### Scoreboard (Data Integrity)

- **Data Match:** The Scoreboard will capture din on valid writes (wr_en && !full) and push it to a reference queue. It will pop from the queue on valid reads (rd_en && !empty) and compare the expected value against dout.

- **Order Match:** The FIFO strictly mandates First-In-First-Out. The Scoreboard must flag a fatal error if the expected value matches a *different* index in the queue, indicating an out-of-order execution.

## 3. Coverage Plan

**Variables to Sample:**

- **Control Signals:** rstn, rd_en, wr_en, full, empty.
- **Fill Levels:** empty (0), near_empty (1), mid_range, near_full (DEPTH - 1), and full (DEPTH).

**Critical Crosses (Corner Cases):**

- **Cross 1: Overflow/Underflow.** Cross wr_en with full to catch write attempts to a full FIFO. Cross rd_en with empty to catch read attempts from an empty FIFO.
- **Cross 2: Simultaneous Access.** Cross rd_en, wr_en, and fill_level. It is crucial to hit the concurrent read/write logic at the exact boundaries: when the FIFO is completely full, and when the FIFO is completely empty.
- **Cross 3: Reset Interruption.** Cross rstn with full and empty to ensure the design handles hard resets appropriately at maximum and minimum stress states.